How to run GEMC and the CLAS12 reconstruction on the JLab farm for an exclurad_py sample

V. Kubarovsky (notes written with Claude, September 2026)

24 September 2026

This is the recipe that was used on 23 September 2026 to simulate 200 000 exclurad_py π^0 events at 6.535 GeV with the RG-K configuration: 80 slurm jobs, three hours, everything from cvmfs. It is written for someone who has never launched GEMC. Every command was run as shown. The scripts live in the exclurad_py repository under `production/farm/`; this note explains them line by line.

1 What you need

- A JLab account with `ifarm` (ssh `ifarm2402`), the farm (slurm, account `clas12`, partition `production`), and the Lustre disk `/volatile`.
- A LUND file of generated events. Anything that writes the standard 10-column header and 14-column particle lines works; exclurad_py, aao_gen, dvcsgen all do.
- Nothing to install. GEMC 5.11, coatjava 11.1.0 and the configuration files (gcards, yaml) are on cvmfs and are loaded with `module`.

Do not run anything heavy on the ifarm login node: processes started there, even inside tmux, are killed when your last ssh session closes. Everything below goes through `sbatch`.

2 A work area on /volatile

Your home directory has a small quota; a single 10 M-event ntuple does not fit. Use

```
mkdir -p /volatile/clas12/users/$USER/gemc_test
mkdir -p /volatile/clas12/users/$USER/gemc_prod/{chunks,logs,out,dst}
```

Files on `/volatile` are deleted automatically after a few weeks of no access: copy what you want to keep to `/work` or to your own machine.

3 Prepare the LUND file

GEMC reads LUND directly, but it **exits silently, producing an output file of a few kB with zero events, if the file contains any line starting with #**. exclurad_py writes its provenance as such lines at the top of every file. Strip them, and split the sample into chunks of 2500 events, which keeps one farm job under three hours:

```
grep -hv '^#' /path/to/lund/*.lund > all.lund
python3 production/farm/split_lund.py all.lund 2500 /volatile/clas12/users/$USER/gemc_prod/
    chunks/
```

The splitter writes `lund_001.dat`, `lund_002.dat`, ... It reads the header of each event to know how many particle lines follow, so it never cuts an event in two.

The vertex columns of the LUND are used by GEMC as they are when they are non-zero. The exclurad_py cards put the vertex in $-5.5 < z < -0.5$ cm, which is the 5 cm cell centred at -3 cm of the fall-2018 and spring-2019 gcards; do not ask GEMC to randomize the vertex on top of that.

4 Which configuration, and which software versions

The gcard (geometry, fields, digitization) and the yaml (reconstruction services) come as a pair from `clas12-config` on cvmfs:

```
/cvmfs/oasis.opensciencegrid.org/jlab/hallb/clas12/sw/noarch/clas12-config/prod/gemc/<gemc
    version>/<name>.gcard
/cvmfs/oasis.opensciencegrid.org/jlab/hallb/clas12/sw/noarch/clas12-config/prod/coatjava/<
    coatjava version>/<name>.yaml
```

The authoritative list of what goes together is `setup.json` in the same repository (`https://github.com/JeffersonLab/clas12-config`). For every configuration it gives the torus and solenoid settings that were run, the beam currents and energies of the background files, the target position, and the *approved software pairs*, in chronological order. As of 24 September 2026:

configuration	fields run	beam	target z	approved gemc + coatjava pairs
`rga_spring2018`	tor -1, tor $+1$	35–50 nA, 10594 MeV	-1.94 cm	4.4.2+6.5.6.1; 5.13+11.1.0; 5.14+11
`rga_fall2018`	tor -1, tor $+1$, tor $+1.01$	40–55 nA, 10604 MeV	-3.0 cm	4.4.2+6.5.6.1; 5.13+10.0.7; 5.14+10
`rga_spring2019`	tor -1	50 nA, 10200 MeV	-3.0 cm	4.4.2+6.5.6.1; 5.13+10.0.7; 5.14+10
`rgk_fall2018_FTOn`	tor $+1$	35–45 nA, 7546 MeV	-3.0 cm	4.4.2+6.5.6.1; 5.10+10.0.7; 5.11+11
`rgk_fall2018_FTOff`	tor $+1$	35–60 nA, 6535 MeV	-3.0 cm	4.4.2+6.5.6.1; 5.10+10.0.7; 5.11+11

Solenoid is -1.00 everywhere. How to read this:

- **The first pair (gemc 4.4.2, coatjava 6.5.6.1) is the pass-1 generation**; it predates binary field maps and the AI denoiser and is history.
- **The second pair is what the pass-2 Monte Carlo was made with**: gemc 5.13 with coatjava 10.0.7 for RG-A fall 2018 and spring 2019 (the aao_norad "pass2" productions on `/cache` used it), gemc 5.10 with coatjava 10.0.7 for RG-K. *The coatjava version of the MC must be the one of the data pass you analyse*, because tracking, PID and the calibrations behave differently between 10.x and 11.x; the gemc minor version fixes geometry details and matters less.
- **The last pair is the newest approved one** (gemc 5.14 for RG-A, 5.11 with coatjava 11.1.0 for RG-K); it is what the OSG portal offers by default and what an upcoming pass will use. The RG-K farm run of 23 September used gemc 5.11 with coatjava 11.1.0, i.e. this pair.
- **RG-K has two gcards**: `FTOn` for the 7.546 GeV period and `FTOff` for the 6.535 GeV period, where the forward tagger was replaced by shielding. This changes the material at 2–7 degrees, exactly where the small-angle electrons and their bremsstrahlung live. The 6.535 GeV run of 23 September was made with `FTOn` and should be repeated with `FTOff` before its small-angle numbers are used.

The MC files themselves do not record the versions (only `RUN::config` with run number and field settings), so keep the pair in your directory names, as the OSG output names do. The torus sign is given on the gemc command line (`-SCALE_FIELD="binary_torus, +1.00"`). Negative torus bends electrons toward the beamline (inbending).

5 One test job of 200 events

Put one chunk into the test directory as `lund.dat`, copy `production/gemc_check/hipo_to_ntuple.groovy` next to it, and create `gemc_test.sbatch` (this is `production/farm/gemc_test.sbatch` with `USER` replaced):

```
#!/bin/bash
#SBATCH --job-name=gemc_test
#SBATCH --account=clas12
#SBATCH --partition=production
#SBATCH --time=01:30:00
#SBATCH --mem=6G
#SBATCH --cpus-per-task=1
#SBATCH --output=/volatile/clas12/users/USER/gemc_test/test_%j.log
set -x
cd /volatile/clas12/users/USER/gemc_test
export TMPDIR=/tmp                      # rule 1: before any "module" command
source /etc/profile.d/modules.sh
module use /cvmfs/oasis.opensciencegrid.org/jlab/hallb/clas12/sw/modulefiles
module use /cvmfs/oasis.opensciencegrid.org/jlab/geant4/modules
module load gemc/5.13                   # the pair of your data pass, see the table
module load sqlite/5.13
export RCDB_CONNECTION=mysql://null
module load coatjava/10.0.7
which gemc recon-util                   # both must print a cvmfs path
GC=/cvmfs/.../clas12-config/prod/gemc/5.13/rga_fall2018.gcard      # pass-2 pair for RG-A
YAML=/cvmfs/.../clas12-config/prod/coatjava/10.0.7/rga_fall2018.yaml
gemc -USE_GUI=0 -NGENP=100 -N=200 -INPUT_GEN_FILE="lund, lund.dat" $GC \
     -SCALE_FIELD="binary_torus, -1.00" -SCALE_FIELD="binary_solenoid, -1.00" \
     -OUTPUT="hipo, gemc.hipo" -INTEGRATEDRAW="*" > gemc.log 2>&1
recon-util -y $YAML -i gemc.hipo -o recon.hipo
ls -la
module purge                            # rule 3: before groovy
export PATH=/group/clas12/packages/groovy/4.0.3/bin:$PATH
/group/clas12/packages/coatjava/9.0.2/bin/run-groovy hipo_to_ntuple.groovy recon.hipo 1000
    test.f32
```

Submit and watch:

```
sbatch gemc_test.sbatch                 # prints "Submitted batch job 12345678"
squeue -u $USER                         # PD = waiting, R = running
tail -f /volatile/clas12/users/$USER/gemc_test/test_12345678.log
sacct -j 12345678 -X --format=State,Elapsed,MaxRSS
```

Expect eight to ten minutes: gemc about 1.4 to 1.9 s per event, recon-util about 1 s per event. **The one check that matters:** `gemc.hipo` must be several MB. A `gemc.hipo` of about 5 kB means gemc read no events; the cause is almost always a `#` line in the LUND. `recon.hipo` is then about 40 kB per event, and `test.f32` 200 bytes per event.

6 The production array

`production/farm/gemc_array.sbatch` is the same script for a slurm array: task i takes `chunks/lund_00i.dat`, runs gemc with `-N=2500` and recon-util in its own work directory, filters the reconstruction output into a DST with only the banks an analysis needs (`RUN::*, REC::*, RECFT::*, MC::*`), writes the ntuple,

and deletes the intermediate files. Edit the same four things as in the test, gcard, yaml, torus sign, `USER`, and the array size:

```
#SBATCH --array=1-80        # 80 x 2500 = 200 000 events
sbatch gemc_array.sbatch
squeue -u $USER -h | wc -l  # how many still queued or running
```

Each task takes 1.5 to 1.8 hours; the 80 of 23 September all ran at once and were done in three hours. Outputs: `out/pi0gemc_NNN.f32` and `dst/pi0gemc_NNN.hipo`. Failures are visible with `sacct -u $USER -name=pi0gemc -X -format=JobID,State,Elapsed`; a task can be rerun alone with `sbatch -array=17 gemc_array.sbatch`.

Budget: about 2.2 core-seconds per event for gemc plus reconstruction, so one million events are 600 core-hours, 400 tasks, half a day to a day of farm time depending on the queue.

7 Getting results

The flat ntuple written by each task has 50 float32 per event: the generated four-vectors of e', p and the two photons, and for each of them the reconstructed particle that `MC::GenMatch` assigns (pid, charge, status, match quality, momentum), plus the counts of identified electrons, protons and photons. Concatenate and plot on your own machine:

```
cat /volatile/clas12/users/$USER/gemc_prod/out/*.f32 > sample.f32
scp ifarm2402:/volatile/clas12/users/$USER/gemc_prod/sample.f32 .
python3 production/gemc_check/plot_kinematics.py sample.f32 plots/mysample --ebeam 10.6
python3 production/gemc_check/boundaries.py plots/mysample mysample=sample.f32@10.6
```

The first gives generated and reconstructed spectra, efficiencies and the yield table; the second the acceptance maps and percentile tables. For any other hipo file (a DST from OSG, an official production on `/cache`) the ntuple is made with

```
export PATH=/group/clas12/packages/groovy/4.0.3/bin:$PATH
/group/clas12/packages/coatjava/9.0.2/bin/run-groovy production/gemc_check/hipo_to_ntuple.
    groovy file.hipo 10000000 out.f32
```

which reads about 20 000 events per second. Reference yields from the official RG-A aao_norad MC at 10.6 GeV: identified $e'p$ in 5.6 % of generated events inbending and 41 % outbending; $e'p\gamma\gamma$ in 0.9 % and 3.7 %.

8 Troubleshooting, in the order the problems were met

1. `module load` prints *Module ERROR: couldn't create error file ... no space left on device* and `which gemc` finds nothing. The slurm per-job `TMPDIR` is not writable for the module system. Fix: `export TMPDIR=/tmp` before the first `module` command.

2. `module load` inside a pipe (`module load x | grep ...`) loads nothing: the environment change dies with the subshell. Never pipe it.

3. gemc runs for a minute, prints *Exiting*, and `gemc.hipo` is 5 kB: comment lines in the LUND. Strip `#` lines.

4. recon-util writes nothing when the output file already exists from a previous attempt: `rm -f recon.hipo` first (the array script does).

5. `run-groovy` fails with *Unsupported class file major version 65*: java 21 from the coatjava module is in the PATH. `module purge` before groovy; the system java 17 is fine.

6. A job started on the login node dies at 18:29 without a message: the last ssh session closed. Use sbatch.

7. *No space left on device* when writing in your home: the home quota. Work on `/volatile`.

8. **bg-merger and recon-util die with *No native library found for os.name=Linux, os.arch=x86_64*** (2026-09-25, cost: 200 jobs that had already spent 3.3 hours each in gemc). `/tmp` is mounted `noexec` on the compute nodes. sqlite-jdbc unpacks `libsqlitejdbc.so` into `java.io.tmpdir` and `dlopen`s it; the unpacking succeeds and the load does not. **This does not reproduce on ifarm2402**, where the identical command with the identical modules works – debugging it on the login node only misleads; test with a short sbatch job. Fix: give the JVM a temp directory on `/volatile`. `bg-merger` and `recon-util` are wrappers that call `java` with hard-coded flags and honour no `JAVA_OPTS`, so `JDK_JAVA_OPTIONS` (Java 9+) is the only way in:

```
export TMPDIR=$W
export JDK_JAVA_OPTIONS="-Djava.io.tmpdir=$W -Dorg.sqlite.tmpdir=$W"
...
unset JDK_JAVA_OPTIONS   # before run-groovy, which uses its own java
```

9. **`[ -s merged.hipo ]` is not a guard.** A header-only hipo is 336 kB and passes it. The job then feeds garbage to recon-util and reports a reconstruction failure for what was a merge failure. Every stage wants a real threshold: `[ $(stat -c %s f) -lt 10000000 ] && exit 1`.

10. **Do not delete `gemc.hipo` until the DST exists.** Three hours of gemc must never be thrown away by a failure in a step that takes seconds. On failure the work directory survives and `post_array.sbatch` resumes from it.

11. **Never reuse a staging directory between campaigns.** On 2026-09-25 a transfer through `/tmp/radstage` – the same path an earlier campaign had used, with the same file names – left 63 of 200 files from the previous day. The event count matched exactly (both campaigns were a million), and only sha256 showed it. Verify a transfer by checksum against the source, never by event count alone, and `rm -rf` the staging directory first.

12. A transfer that runs at 0 kB/s is hung, not slow. On 2026-09-25 an rsync sat for 2 h 14 min having moved 865 of 869 MB, while the link measured 97 MB/s; killing it and starting over took 8 seconds. Sample the destination size twice before waiting on it.

9 Two stages, and how to resume the second

Generation and simulation are separate arrays on purpose (`gen_array.sbatch` then `sim_array.sbatch`): a 12-hour job waits for a rare gap in a backfill queue full of 2–5 hour jobs, and a gemc crash should not throw away two hours of generated events.

`sim_array.sbatch` keeps its work directory when a post-gemc stage fails, so `post_array.sbatch` can redo only the cheap part from the existing `gemc.hipo`:

```
sbatch --array=1-200 --job-name=post_born --time=02:00:00 post_array.sbatch born
```

It refuses to start without a `gemc.hipo` of at least 1 MB, and deletes the work directory only once the DST is written. On 2026-09-25 this recovered 660 core-hours of gemc that the sqlite failure above would otherwise have cost: the background merge takes 30 seconds against gemc's 3.3 hours.

10 Wall-clock, measured – and it varies by a factor two

With 50 nA background merged over the eleven detectors, 5000 events per task, measured over 400 tasks on 2026-09-25/26:

stage	fastest	slowest
gemc	11 945 s (3.3 h)	24 487 s (6.8 h)
bg-merger	28 s	50 s
reconstruction	5946 s (1.65 h)	6748 s (1.87 h)
full chain	5.0 to 8.7 hours	

The gemc time is **not reproducible to better than a factor two** across farm nodes – the same 5000 events took 3.3 h on one and 6.8 h on another. Budget for the slow end:

- full chain (`sim_array.sbatch`): `-time=10:00:00`. Five hours killed 30 of 200 jobs *during gemc*, which is the expensive way to lose them; seven hours killed 29 of 200 during reconstruction, which is the cheap way, because `gemc.hipo` survives and `post_array.sbatch` finishes them in two hours.
- catch-up (`post_array.sbatch`): `-time=04:00:00`. Two hours gave an 8 % margin over a 1.9 h reconstruction and lost 41 jobs at the limit.

Memory, over 679 steps of the no-background chain and 553 of the chain with bg-merger: **nothing has exceeded 2.7 GB**, median 1.75–1.93 GB. `-mem=6G` is the number this note used to carry from RG-K, and it is nearly a factor three too large. Ask for 4 G. That is the bigger backfill lever of the two, because the gap the scheduler has to find scales with the request.

A pending job's request can be shrunk in place, which keeps the job id and so the queue position, unlike cancel-and-resubmit:

```
scontrol update jobid=NNN MinMemoryNode=4096   # MEGABYTES -- "4G" is rejected
scontrol update jobid=NNN TimeLimit=03:00:00   # reducing is allowed
```

Slurm will not *extend* a running or pending job (*Access/permission denied*), so the time limit has to be right when you submit; memory can be tightened later, once the first tasks have finished and `sacct -format=MaxRSS` has something to say.

And when you tighten it, check that your measurement is not censored by the limit you are about to change. On 2026-09-26 the catch-up array's completed tasks topped out at 01:53 and that looked like the tail; it was the 2 h wall, which had already killed 41 tasks. The uncensored sample from the 4 h array has a median of 1.93 h, a 95th percentile of 3.07 h and a worst case of 3.77 h — so a 3 h limit chosen from the censored figure loses 10 % of the array, not the 2 % the censored one implies. A completed-jobs-only distribution says nothing about the region beyond the limit, which is the region the decision turns on.

Two distributions of the same chain, measured the same day, agree in shape once the censored one is replaced: with bg-merger, median 1.93 h, 95th 3.07, worst 3.77; without it, median 1.80, 95th 2.72, worst 3.65 over 667 tasks. The extra stage shifts the whole distribution up rather than damping its spread.

How hard to pull the lever depends on what a failure costs you, not on the distribution alone. The same 3 h limit loses about 10 % of one array and 2 % of the other, but the decision differed for a second reason: one run had a watcher resubmitting failures at 4 h automatically, so a loss was a delay; the other had none, so every task that died had to be noticed and resubmitted by hand. Cut close to the tail when retry is automatic; leave margin when it is not.

One million events is 200 tasks and roughly 700 to 1400 core-hours.

11 OSG instead of the farm

The CLAS12 MC portal (`https://gemc.jlab.org/web_interface/`) runs the same gemc and coatjava versions from the same configuration files, adds background merging from real data and the denoiser, and delivers the DSTs to `/cache`. It accepts user LUND files. That is the route for a physics sample; the farm route above is for tests, acceptance and resolution studies, and for anything that must run today. The two produce the same banks, so the ntuple and plotting tools apply to both.